- *A* `PageResult` *object*—This causes the associated Razor view to generate an HTML response.
- *Nothing (the handler returns* `void` *or* `Task`*)*—This is the same as the previous case, causing the Razor view to generate an HTML response.
- *A* `RedirectToPageResult`—This indicates that the user should be redirected to a different page in your application.

These are the most common results for Razor Pages, but I describe some additional options in section 15.4.

It's important to realize that a page handler doesn't generate a response directly; it selects the type of response and prepares the data for it. For example, returning a `PageResult` doesn't generate any HTML at that point; it merely indicates that a view should be rendered. This is in keeping with the MVC design pattern in which it's the *view* that generates the response, not the *controller*.

TIP The page handler is responsible for choosing what sort of response to send; the *view engine* in the MVC framework uses the result to generate the response.

It's also worth bearing in mind that page handlers generally shouldn't be performing business logic directly. Instead, they should call appropriate services in the application model to handle requests. If a page handler receives a request to add a product to a user's cart, it shouldn't manipulate the database or recalculate cart totals directly, for example. Instead, it should make a call to another class to handle the details. This approach of separating concerns ensures that your code stays testable and maintainable as it grows.